

Index Search

Overview

This module handles indexing and searching of JSON documents in collections. Documents are indexed by keywords for full-text search. Supports complex queries including boolean operators, phrases, wildcards, and relevance ranking.

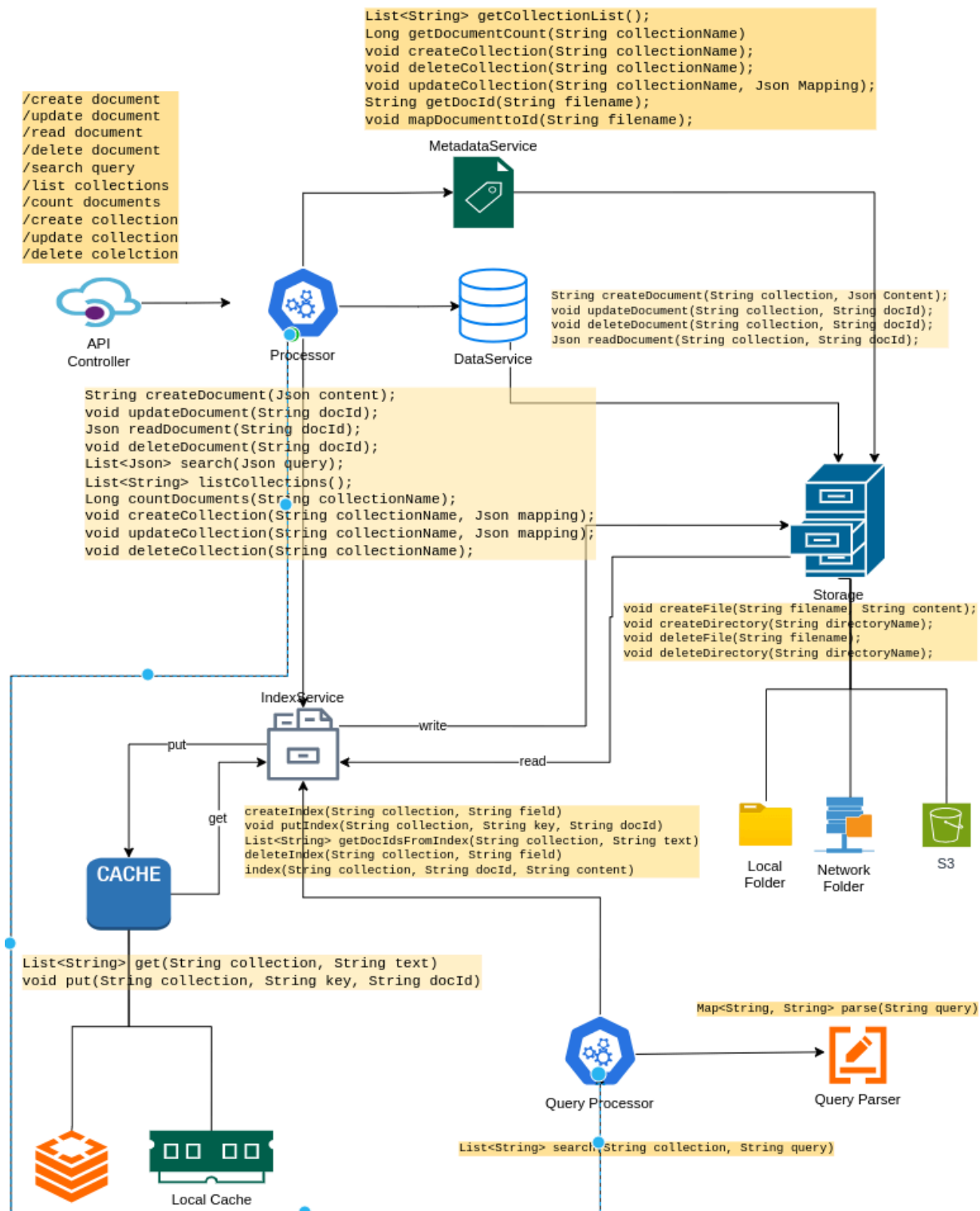
Functional Requirements

- ☒ Index any text/string field in JSON documents
- ☒ Support full-text indexing with tokenization, lowercasing, stemming, stop-word removal
- ☒ Allow configurable indexed fields per collection
- ☒ Index new documents automatically on create
- ☒ Update index synchronously or asynchronously on document update
- ☒ Remove index entries on document delete
- ☒ Delete entire collection index on collection deletion
- ☒ Support keyword search
- ☒ Support boolean queries: must / should / must_not (AND/OR/NOT)
- ☒ Support pagination (from/size)

Non-Functional Requirements

- ☒ Index latency < 200 ms per average document (sync mode)
- ☒ Search latency < 150 ms for typical queries (< 10 terms)
- ☒ Support ≥ 1 million documents per collection
- ☒ Support ≥ 100 concurrent search requests
- ☒ Eventual consistency (async indexing allowed)
- ☒ Index size ≤ 30% of raw document size
- ☒ Graceful degradation on cache miss
- ☒ No data loss on index update failure (retry + fallback)
- ☒ Query parsing rejects invalid syntax (400 response)
- ☒ Support horizontal scaling of index/search nodes
- ☒ Zero-downtime collection re-indexing (future)

Design Diagram :



4. Components

Storage

- Manages storage of files and directories
- Three variants: Local Folder storage, Network Folder storage, S3 Storage

DataService

- Manages document create, update and delete
- Depends on Storage to create files

MetadataService

- Manages maintenance of collections.
- Depends on Storage to create required files and directories.

IndexService

- **Responsibilities:**
 - Create/update/delete indexes per collection.
 - Index JSON documents: Extract text from specified fields (configurable via MetadataService).
 - Tokenization: Lowercase, remove stop words, stem (e.g., Porter stemmer).
 - Inverted Index Structure: Map<Keyword, List<DocID, Positions>>.
- **Methods:**
 - createIndex(collectionName)
 - putIndex(collection, docId, content)
 - getDocIds(query)
 - deleteIndex(collection, docId)
- **Backend:** In-memory for small scale; integrate with Lucene/Elasticsearch for production.

Cache

- Manages faster retrieval of docIds from word.
- Two variants: Local in-memory cache, External key-value pair service like Redis

QueryProcessor

- **Responsibilities:**
 - Parse sql query
 - Build query map: Support boolean logic, phrases, wildcards.
 - Execute against IndexService: Retrieve docIds
 - Post-process: Filter, sort, paginate.
- **Methods :**

- search(collection, query): Returns list of docIDs.
- **Query Language:**
 - Structured Query Language
 - Example: Select name from personCollection where dob >= '2026-01-01'

Processor

Process all the API requests

Data Flows

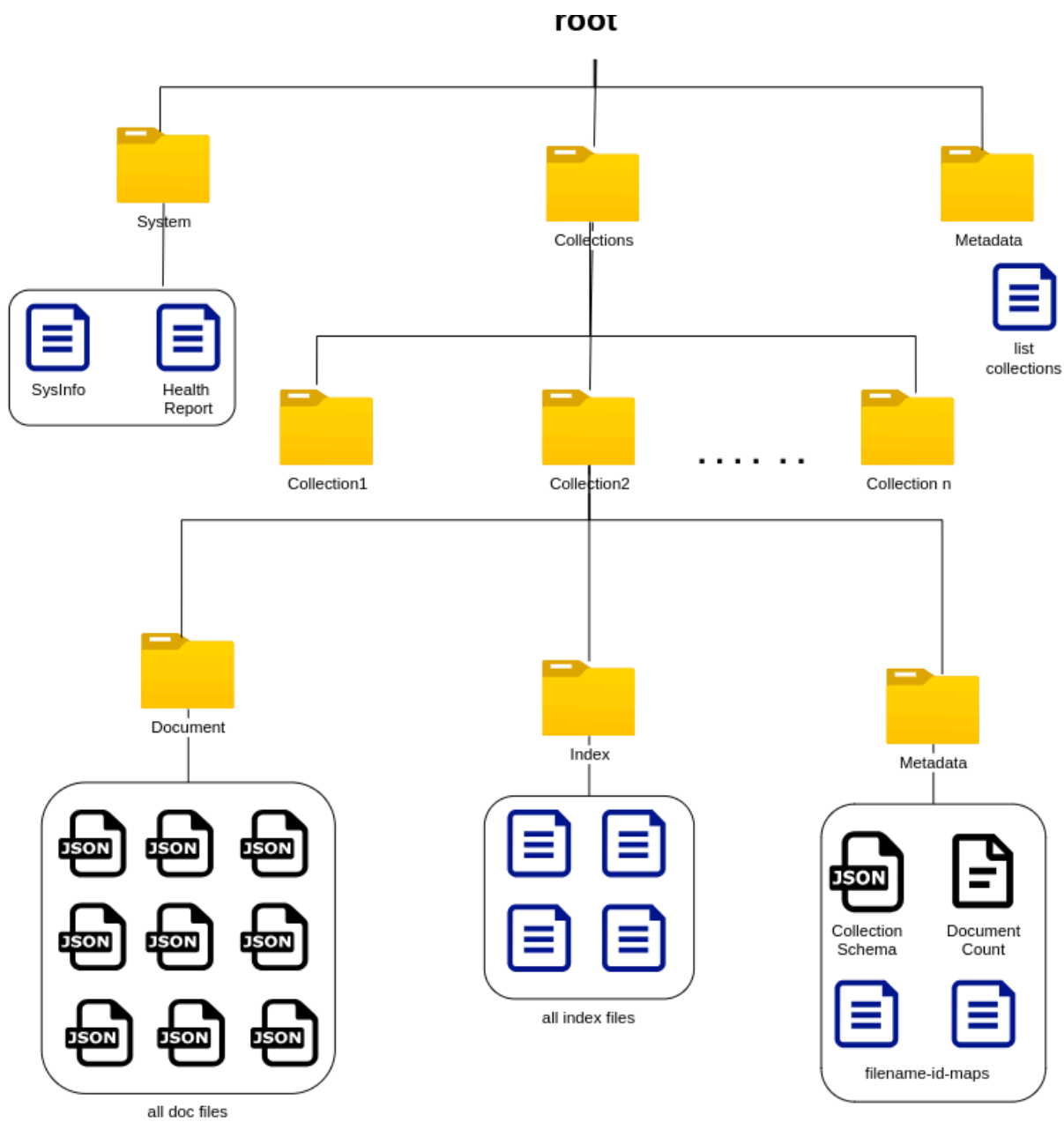
Indexing Flow

1. DataService calls IndexService.putIndex on create/update.
2. Parse JSON: Tokenize configurable fields (default: all strings).
3. Update inverted index: Add keyword → (docId, positions).
4. Cache invalidates for affected keys.
5. MetaDataService updates index stats (doc count, last indexed).

Search Flow

1. API Controller receives search request.
2. Processor validates via MetaDataService (collection exists, query syntax).
3. QueryProcessor parses JSON query.
4. IndexService.getDocIds: Intersect/union sets based on operators.
5. Rank results: Compute scores.
6. DataService fetches full docs from Storage/Cache.
7. Return paginated JSON list.

Folder Structure



Index Structure

- **Inverted Index:**
 - Stored in the Storage backend (e.g., files for LocalStorage).
 - Format: Serialized map (e.g., JSON or binary).
 - Sharding: By collection; optional by keyword hash for large indexes.
- **Metadata:** Per collection, stores indexed fields, analyzers (tokenizer config).

For each collection, there will be one index per indexed field aka key field.

If a collection schema defines like this:

```
{
  "name": "people",
  "keys": [ { "field": "name", "type": "text" }, { "field": "city", "type": "text" } ],
  "data": [ { "field": "dob", "type": "text" }, { "field": "address", "type": "text" } ]
}
```

Then there will be two indexes created for collection "people", as it has two indexed fields: "name" and "city".

There will be a file holding a character map. This will work as a Trie.

Any new word added, will be added to the trie character wise. Each line represents the corresponding character position. Each entry will link to the index file through a wordId.

Index a word : butterfly

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	
	■																									b
																				■						u
																			■							t
																			■							t
				■																						e
																	■									r
						■																				f
											■															l
																								■		y

exists: true
filename: indexfilexyz
position : 55

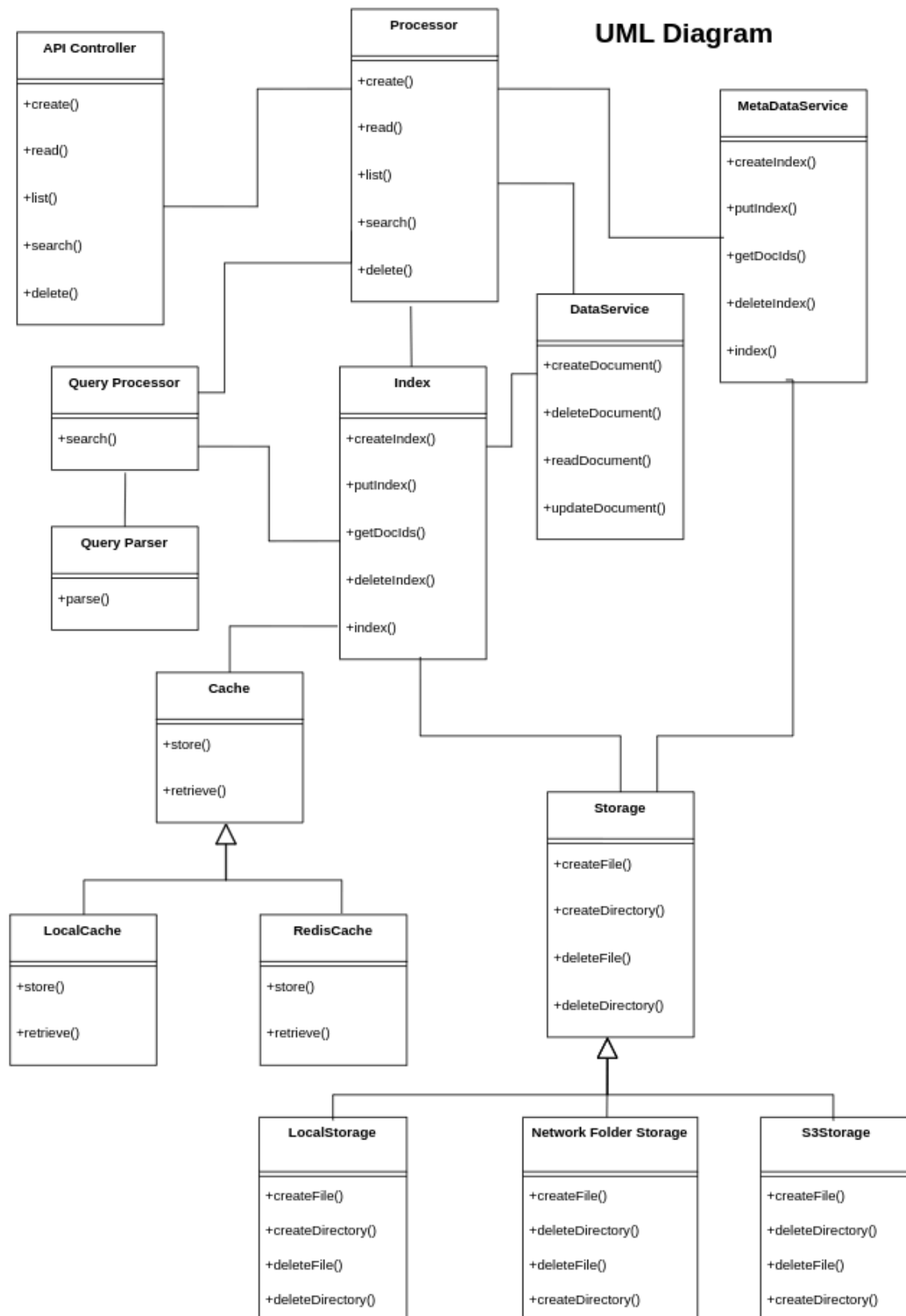
indexfilexyz		
wordId	docIdList	
143BA22	doc1, doc5	55

Trie Node Memory mapped file

Offset	Size	Content	Example value
0	4	Magic number / version	0x54524945 ("TRIE")
4	8	Root node offset	16
12	8	Total file size (for validation)	8192


```
// Node structure (repeated many times)
Node N:
    offset      1 byte      is_end_of_word (0 or 1)
    offset+1    4 bytes      bitmap (32 bits = 4 bytes, e.g. supports a-z +
6 special)
    offset+5    1 byte      child_count (number of set bits in bitmap)
    offset+6    child_count × 8 bytes  child offsets (file positions of
child nodes)
    offset+6+... (only if is_end == 1)  postings pointer:
                                     1 byte      filename length
                                     N bytes      filename (e.g. "postings_3.bin")
                                     1 byte      \0 terminator
                                     8 bytes      offset in postings file (uint64)
```


UML



Pseudocode

Query Processor

```
function search(queryStr):  
    // QueryTree (collectionName, field, conditions, group, order)  
    parsedQueryMap = parseQuerySql(queryStr)  
  
    if parsedQuery invalid:  
        return error "Invalid query syntax"  
  
    if not MetaDataService.collectionExists(collectionName):  
        return error "Collection not found"  
  
    fieldDocIdMap = empty  
    for each key field check:  
        docIdList = getList(collectionName, keyField)  
        fieldDocIdMap.add(key, docIdList)  
    for each NOT clause:  
        docIdList = negation(docIdList)  
    for each AND clause:  
        docIdList = intersection(docIdList)  
    for each OR clause:  
        docIdList = union(docIdList)  
  
    // Apply conditions/filter  
    filteredDocList = applyFilters(docIdList, parsedQueryMap)  
  
    // Sort by order fields  
    sortedDocList = sort(filteredDocList, parsedQueryMap)  
  
    return sortedDocIdList
```

TODO : Apply Pagination

Open Questions

How is the index managed?

Indexes are stored in files. When receiving a keyword, it is required to check all the field specific index whether the keyword is present in the index. If index files are in parts then need to manage multiple files.

When a doc is deleted all of the keywords of the doc need to be searched from the index and the docId needs to be removed.

How can this be faster? it is a sequential file. We need to do something better.

1. LSM Tree + Memory Mapped File
2. RocketDB
3. Trie + Memory mapped file

How to maintain filename to docId mapping in metadata files?

A text file that contains the map of filename to docId.

Considering this list will be huge as a lot of documents will be inserted in a collection.

Every time a new document is created its id needs to be added to the list.

Every time a document is deleted, it needs to be removed from the list.

The list is stored in a single file and will be a huge list. can be stored in parts. Maintenance becomes tough with part files. How to improve here?

How to maintain a list of collections?

Maintain a text file with all the collection names in it.

Then if a new collection is created, it will be appended at the end of content. Now if we want to delete a collection, its name should be removed from the list also. Sequential lists maintain problems.

If we need to find a collection name, it will be a sequential search.

Can we do better than this?

Although we hope the list of collections will not grow massive, it is still fine.